TALLER DOCKER

Contenerizacion, Seguridad de Runtime y Estres de cgroups

Estudiante	Felipe Narvaez Gomez
ID estudiantil	908404
Proyecto contenerizado	hotelspringboot (Spring Boot, clase HotelApplication)
Imagen	hotelspringboot:1.0.0
Fecha	20/10/2026

1. Objetivo del taller

El taller busca empaquetar un backend Spring Boot en una imagen Docker de produccion que sea **ligera**, **segura** y con **cuotas estrictas de hardware**. Los riesgos que se quieren evitar son dos: que el contenedor corra con privilegios de root, y que la JVM consuma toda la memoria del servidor hasta que el sistema operativo termine el proceso. La evidencia se obtuvo ejecutando comandos de diagnostico reales sobre el daemon de Docker local, no con teoria.

2. Reto 1: Construccion multietapa

Se escribio un `Dockerfile` de dos etapas y un `.dockerignore`. Comando usado para construir la imagen (el Dockerfile esta en la carpeta del proyecto, un nivel arriba de la carpeta de entrega):

```
docker build --build-arg STUDENT_ID="908404" -t hotelspringboot:1.0.0 -f ..\Dockerfile ..
```

Decisiones de diseno y su justificacion

Decision	Justificacion
Etapa builder con maven:3.9.6-eclipse-temurin-17-alpine	Compila el proyecto con Maven y JDK 17. Esta etapa se descarta: nada de ella llega a la imagen final.
Copiar solo pom.xml y ejecutar mvn dependency:go-offline -B antes de copiar src/	Docker guarda cada instruccion como una capa en cache. Las dependencias cambian poco y el codigo cambia mucho; asi, al modificar src/ no se vuelven a descargar las dependencias.
ARG STUDENT_ID (re-declarado en la etapa 2)	Inyecta el ID en tiempo de construccion. Los ARG no se heredan entre etapas, por eso se declara de nuevo para poder usarlo en ENV.
mvn package -DskipTests	Genera el .jar ejecutable sin correr pruebas, como exige el enunciado.
Etapa runtime con eclipse-temurin:17-jre-alpine	Solo incluye el JRE sobre Alpine: menor tamano y menor superficie de ataque que un JDK completo.
Usuario sensor_user (UID 10001) en el grupo sensorgroup	Evita ejecutar la aplicacion como root. Un UID alto no coincide con usuarios reales del host.
COPY --from=builder solo del .jar y chown de /app	En la imagen final unicamente esta el artefacto necesario, y pertenece al usuario sin privilegios.
ENV OPERATOR_ID=${STUDENT_ID}	Deja el ID del estudiante dentro de la imagen, verificable con docker exec ... env.

Decision	Justificacion
ENV JAVA_OPTS con UseContainerSupport y MaxRAMPercentage	La JVM dimensiona el heap segun el limite del contenedor (cgroup) y no segun la RAM total del servidor.
USER sensor_user antes del ENTRYPOINT	El proceso java arranca ya sin privilegios.

La imagen se construyo correctamente. Su identificador (hash SHA-256) se reporta en la tabla de identidad de la seccion 5.

3. Reto 2: Auditoria forense

3.1 Desacoplamiento de capas

```
docker history --human --format "table {{.Size}}\t{{.CreatedBy}}" hotelspringboot:1.0.0
SIZE      CREATED BY
0B        ENTRYPOINT ["sh" "-c" "exec java $JAVA_OPTS ...]
0B        USER sensor_user
0B        EXPOSE [8080/tcp]
0B        ENV JAVA_OPTS=-XX:+UseContainerSupport -XX:M...
0B        ENV OPERATOR_ID=908404
47.7MB    RUN |1 STUDENT_ID=908404 /bin/sh -c chown -R...
47.7MB    COPY /build/target/*.jar /app/app.jar # buil...
8.19kB    WORKDIR /app
41kB      RUN |1 STUDENT_ID=908404 /bin/sh -c addgroup...
0B        ARG STUDENT_ID=908404
...       (capas base de eclipse-temurin:17-jre-alpine: JRE de 141MB, Alpine 9.08MB)
```

Analisis: el historial solo muestra las capas del JRE, la creacion del usuario, el .jar y la configuracion. No aparece ninguna capa de Maven ni del codigo fuente: quedaron en la etapa *builder*, que se descarta. La capa del chown pesa lo mismo que el .jar (47.7MB) porque al cambiar el propietario Docker duplica el archivo en una nueva capa; una mejora seria usar `COPY --chown` para evitar esa duplicacion.

3.2 Aislamiento de privilegios

```
docker top hotelspringboot_active -o pid,user,comm
PID                 USER                COMMAND
1654                10001               java

docker exec hotelspringboot_active id
uid=10001(sensor_user) gid=10001(sensorgroup) groups=10001(sensorgroup)
```

Analisis: el proceso java corre con el UID 10001. El host solo conoce el numero, por eso *docker top* lo muestra como 10001; el comando *id* dentro del contenedor confirma que ese UID corresponde a **sensor_user**. El proceso no es root.

3.3 Inmutabilidad de runtime

```
docker exec hotelspringboot_active touch /etc/crontab_fake
touch: cannot touch '/etc/crontab_fake': Permission denied
```

Analisis: la escritura fue rechazada porque /etc pertenece a root y sensor_user no tiene permiso de escritura. Si un atacante lograra ejecutar comandos a traves de la aplicacion, no podria dejar scripts persistentes en directorios del sistema.

3.4 Artefactos de desarrollo

```
docker exec hotelspringboot_active sh -c "command -v mvn"
(sin salida, codigo de salida 1)

docker exec hotelspringboot_active sh -c "find / -name '*.java' 2>/dev/null"
(sin salida)
```

Analisis: Maven no esta instalado y no existe ningun archivo .java en la imagen final. El build multietapa dejo fuera las herramientas de compilacion y el codigo fuente, lo que reduce el tamano y limita lo que un intruso podria aprovechar. (Nota: se uso *command -v mvn*, equivalente a *which mvn*, que no siempre esta disponible en Alpine.)

4. Reto 3: Calibracion de cgroups y OOM

4.1 Colapso por memoria con 48 MB

```
docker inspect hotelspringboot_crash --format="Status: {{.State.Status}} |
  ExitCode: {{.State.ExitCode}} | OOMKilled: {{.State.OOMKilled}}"
Status: exited | ExitCode: 137 | OOMKilled: true
```

Analisis: con solo 48 MB el kernel de Linux termino el contenedor. El codigo 137 equivale a 128 + 9, es decir, la senal SIGKILL, y *OOMKilled: true* confirma que la causa fue exceder el limite de memoria del cgroup. Spring Boot con Hibernate necesita mucha mas memoria que esa cuota.

4.2 Calibracion del limite operativo

Limite -m	Inicio OK	Tiempo de arranque	Observaciones
48m	No	No inicio	OOMKilled, ExitCode 137
256m	Si	22.617 s	Sigue en ejecucion; puerto del host 8081
384m	No probado	-	-
512m	No probado	-	-

```
docker logs --timestamps hotelspringboot_active | findstr "Started HotelApplication"
2026-10-20T22:12:16.346340031Z 2026-10-20T22:12:16.345Z  INFO 1 --- [hotel-reservas-app]
  [main] com.hotel.Hotel.HotelApplication : Started HotelApplication in 22.617 seconds
  (process running for 25.844)
```

CPU %	MEM USAGE / LIMIT	MEM %
0.19%	163.9MiB / 256MiB	64.02%

```
docker stats --no-stream hotelspringboot_active
CONTAINER ID   NAME                    CPU %   MEM USAGE / LIMIT    MEM %   NET I/O          PIDS
e3ebe3660c49   hotelspringboot_active   0.19%   163.9MiB / 256MiB    64.02%  28.5kB / 33.3kB  32
```

Analisis: con 256 MB el contenedor inicia y se mantiene estable: usa 163.9 MiB (64.02% del limite), con CPU casi en reposo (0.19%) y 32 procesos/hilos. Queda un margen de unos 92 MiB para picos de carga. El limite de 256 MB es el **minimo exitoso entre los valores probados**.

Limitacion que debe tenerse en cuenta: el arranque tardo 22.617 segundos, por encima del objetivo de 15 segundos del enunciado, y no se probaron los limites de 384 MB y 512 MB. Por tanto, el criterio de tiempo no se cumplio con 256 MB. Es esperable que un limite mayor reduzca el tiempo de arranque, ya que la JVM y el recolector de basura disponen de mas margen durante la carga inicial de

Spring y Hibernate, pero esto debe confirmarse ejecutando esas pruebas.

5. Tabla de identidad

Item	Comando	Salida
Hash de la imagen	docker inspect --format="{{.Id}}" hotelspringboot:1.0.0	sha256:d54de4174cfa4840e470533741896e237ffe313b76e78c08b8cf7a314dd2e416
Variable inyectada	docker exec hotelspringboot_active env | findstr OPERATOR_ID	OPERATOR_ID=908404
Consumo final calibrado	docker stats --no-stream hotelspringboot_active	CPU 0.19% | 163.9MiB / 256MiB | 64.02%

6. Conclusiones

Se logro una imagen **ligera** (solo JRE y el .jar, sin Maven ni codigo fuente), **segura** (proceso java como sensor_user, sin permisos de escritura en el sistema) y con un **consumo de memoria conocido** (163.9 MiB en reposo bajo un limite de 256 MiB). Se verifico que un limite demasiado bajo (48 MB) provoca la terminacion del contenedor por el kernel con OOMKilled y codigo 137, lo que demuestra por que es necesario calibrar los limites antes de llevar el servicio a una orquestacion multicapa, donde cada servicio declara su cuota de recursos.

Como mejoras: usar `COPY --chown` para no duplicar el .jar en una capa extra, y completar las pruebas con 384 MB y 512 MB para hallar el limite que cumpla el arranque en menos de 15 segundos.